Studio Nx

JS 142 — 트랙 순서 드래그

2026-10-14

JS 142 — 트랙 순서 드래그 배선 (`initTrackReorder`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js`
작업	S1-06 타임라인: 스물여덟 번째 작업 (FR-207 트랙 순서 드래그)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-06-timeline`
다음	트랙 이름·색 변경(FR-207)과 오디오 클립 배치 배선을 이어서 진행합니다

1. 하는 일

트랙 머리의 ⠿ **그립**을 위아래로 끌어 순서를 바꿉니다. 놓으면 `reorderItems()`로 새 순서를 만들고 DOM을 다시 붙인 뒤, `renumberTracks()`로 번호를 다시 매깁니다.

```
1. Lead   ⠿        ← 0번 그립을 아래로 끌어 2번 자리로
2. Strum
3. Drums
─────────
1. Strum
2. Drums
3. Lead           ← 옮기고 번호 다시 매김
```

- 끄는 동안 그 트랙에 `is-reorder-dragging`, 놓일 자리 트랙에 `is-reorder-target`(CSS가 표시)을 붙입니다.
- 그룹 줄(`Low End GROUP`)처럼 번호 없는 줄은 배열에 그대로 있고, 번호정리에서 건너뜁니다.
- 새로 만든 트랙(`createTrack`·`addTrack`)에도 그립을 넣어 새 트랙도 끌 수 있습니다.

2. 코드

```
function initTrackReorder({ lanes, onChange }) {
  const tracksEl = lanes.querySelector('#tracks');
  if (!tracksEl) return;
  let drag = null;
  const items = () => Array.from(tracksEl.children).filter((el) => el.classList.contains('sq-track'));
  const clearMarks = () => items().forEach((el) => el.classList.remove('is-reorder-target'));

  const onMove = (e) => {
    if (!drag) return;
    const list = items();
    let at = 0;
    for (const el of list) { const r = el.getBoundingClientRect(); if (e.clientY > r.top + r.height / 2)
at += 1; }
    const to = Math.max(0, Math.min(list.length - 1, drag.from < at ? at - 1 : at));
    drag.to = to;
    clearMarks();
    const target = list[to];
    if (target && target !== drag.track) target.classList.add('is-reorder-target');
  };
```

```
  function end(commit) {
    if (!drag) return;
    const d = drag; drag = null; cleanup();
    d.track.classList.remove('is-reorder-dragging');
    clearMarks();
    if (!commit) return;
    const list = items();
    const next = reorderItems(list, d.from, d.to);
    let changed = false;
    for (let i = 0; i < next.length; i++) { if (next[i] !== list[i]) { changed = true; break; } }
    if (!changed) return;
    next.forEach((el) => tracksEl.append(el));
    renumberTracks(onChange);
    onChange({ type: 'reorder', trackId: d.track.dataset.trackId, from: d.from, to: d.to });
  }

  lanes.addEventListener('pointerdown', (e) => {
    const grip = e.target.closest('.sq-grip');
    if (!grip || e.button !== 0) return;
    const track = grip.closest('.sq-track');
    if (!track || !tracksEl.contains(track)) return;
    e.preventDefault();
    const list = items();
    drag = { track, from: list.indexOf(track), to: list.indexOf(track) };
    track.classList.add('is-reorder-dragging');
    document.addEventListener('pointermove', onMove);
    document.addEventListener('pointerup', onUp);
    document.addEventListener('pointercancel', onCancel);
  });
}
```

(`cleanup`·`onUp`·`onCancel`은 위 끝맺음과 함께 붙습니다.) **최신 문법으로 쓰면** (공부용)

```
const items = () => [...tracksEl.children].filter((el) => el.classList.contains('sq-track'));
```

최신 문법	실제 코드	뜻
스프레드 `[...children]`	`Array.from(children)`	같은 결과입니다

3. 왜 이렇게

선택	이유
`to` 계산에 `from < at ? at-1 : at`	끄는 항목을 빼면 아래가 한 칸 당겨집니다. 그만큼 보정해야 손이 가리킨 자리에 놓입니다
그립에서만 시작	트랙 머리 빈 곳을 눌러도 드래그가 시작되지 않습니다. 좌우 이동(pan)·클립 끌기와 겹치지 않습니다
`reorderItems()` 재사용	순서 바꾸기 규칙(131)을 이미 시험했습니다. 화면 코드는 그 결과를 DOM에 붙이기만 합니다
안 바뀌면 건너뜀	제자리에 놓으면 번호정리도 알림도 하지 않습니다. 불필요한 저장을 막습니다
놓기 전에는 안 바꿈	미리보기(표시)만 하고, 실제 순서는 놓을 때 한 번 바꿉니다. 되돌리기가 쉽습니다

4. 시험

(1) Node 단위 시험 — 모듈 적재 확인(11개 통과). `reorderItems` 자체는 131에서 15개 통과.

(2) 브라우저 확인(Chrome headless + CDP, 실제 `studioNx.html`) — 그립에 `pointerdown` → `pointermove`(아래로) → `pointerup` 을 흉내 냈습니다.

확인	결과
끄는 동안 `is-reorder-dragging`, 놓일 자리 `is-reorder-target` 표시	✓
0번(Lead)을 2번 자리로 → 순서 `Lead,Strum,Drums → Strum,Drums,Lead`	✓
놓은 뒤 번호가 자리대로 다시 매겨짐(1.Strum·2.Drums·3.Lead)	✓
그룹 줄(번호 없음)은 자리 유지·번호정리에서 건너뜀	✓
놓은 뒤 표시(`is-reorder-*`)가 지워짐	✓